

Cross-Platform Customer Purchase Analysis

Description: Using Python, Pandas, and related libraries, this notebook

1. Extracts Customer Purchase data from 3 data sources, a flat file, a SQLite table, and a JSON Web API.
2. Transforms the extracted data into a consistent, usable high-quality format.
3. Merges the transformed data into a single denormalized DataFrame.
4. Aggregates the merged detail records into a second summary DataFrame.
5. Loads these 2 DataFrames into a data warehouse of 2 MySQL tables that can be queried to find new business information that was not readily available from the 3 separate sources alone.

Purpose: To demonstrate a practical understanding of the Extract-Transform-Load process and competence in its execution.

Goals:

1. To gain experience with SQL databases such as SQLite and MySQL and their interaction with Python/Pandas.
2. To gain experience with accessing data using a Web API.
3. To gain experience with cleansing, integrating, and aggregating data.
4. To gain experience with designing a denormalized Data Warehouse schema.

Author: Chris Indorf

Date: 4/25/2025

Table of Contents

Install and Import Libraries

Extraction

Extract Flat File

Extract SQLite table

Extract from Web API

Transformation

Transform Flat File

Transform SQLite Table

Transform Web API File

Join the 3 Datasets

Aggregate the Joined Dataset into a Summary DataFrame

Loading

Create MySQL Tables

Insert Data into MySQL Tables

Verify the Loading Process

Analytical Query Examples

Summary

Appendix - File and Table Schemas

Install and Import Libraries

```
In [453... '''Install and import libraries
!pip install ratelimit
!pip install mysqlx-connector-python
!pip install mysql-connector-python
!pip install SQLAlchemy '''

import pandas as pd
import sqlite3
from ratelimit import limits, sleep_and_retry
import requests
import json
import mysql
import mysql.connector
from sqlalchemy import create_engine, text
```

Extraction

Extract Flat File

```
In [455... '''Online Sales Data
customer ID, order date, product ID, quantity, price, payment method, shipping address,
and website source (e.g., organic, paid ad). '''

online_df = pd.read_csv('online_sales.csv')
online_df.head()
```

Out[455...

	customer_id	order_date	product_id	quantity	price	payment_method	shipping_address	website_
0	1	2024-03-05	1	2	834.50	PayPal	Address 92	F
1	1	2024-03-05	1	1	915.03	PayPal	Address 28	F
2	1	2024-03-14	1	2	320.26	PayPal	Address 94	F
3	1	2024-04-03	1	1	356.52	Credit Card	Address 2	F
4	1	2024-04-04	1	1	621.00	PayPal	Address 99	C



Extract SQLite Table

```
In [457... '''In-Store Transaction Data
transaction ID, customer ID (if available, otherwise anonymized), transaction date, product ID
store location, and payment method. '''

# Connect to SQLite
instore_con = sqlite3.connect(r'C:\Users\cindo\Documents\Personal\NVCC\Data Science II\ETL\in_
instore_cur = instore_con.cursor()

# Import query into dataframe
instore_df = pd.read_sql_query('SELECT * FROM instore', instore_con)

instore_df.head()
```

```
Out[457...      customer_id  transaction_date  product_id  quantity  price  store_location  payment_method
0             17.0        2024-08-06             2         5     63          Store 3        Debit Card
1            331.0        2025-02-15             5         2    462          Store 14        Credit Card
2            286.0        2024-10-29             1         2    304          Store 2        Debit Card
3            394.0        2024-05-12             2         4    125          Store 20             Cash
4            237.0        2024-09-09             7         5     58          Store 11        Debit Card
```

Extract from Web API

```
In [ ]: # Customer Support Interactions. Load dataframe.
''' JSON WEB API customer ID, interaction date, interaction type (e.g., inquiry, complaint, re
mentioned), and sentiment score (analyzed from text).

There are only 2000 records, with auto-generated integer IDs, starting at id=1 to id=2000. Eac
individually.

There is a rate limit over a sliding window of time for accessing the API. This limit was dete
to be 50 calls per minute. Thus it takes exactly 40 minutes to extract 2000 records.

Here's the URL API endpoint with an example:
api_key (use your name: LASTNAME_FIRSTNAME) and individual ID of a record (1213):
https://api.profammons.com/itd245/customer-support/?api_key=INDORF_CHRIS&id=1213

Handle nested JSON structures and schema variations.
...

@sleep_and_retry
@limits(calls=50, period=60)
def api_call(api_key):
    response = requests.get(api_key)
    if response.status_code != 200:
        raise Exception('API Response: {}'.format(response.status_code))
    return response

support_df = pd.DataFrame()
for id in range(1, 2001):
    api_key = 'https://api.profammons.com/itd245/customer-support/?api_key=INDORF_CHRIS&id='+s
    # print progress indicator
    print(id, end=' ')
    response = api_call(api_key)
    if response.status_code == 200:
```

```
support_df = pd.concat([support_df, pd.json_normalize(response.json())])
else:
    print(f"Error: {response.status_code}")
```

```
In [442... # Display results
support_df.head()
```

Out[442...

	id	customer_id	interaction_date	interaction_type	product_id	sentiment_score
0	1	126	2025-01-18	Feedback	3	0.87
0	2	346	2024-12-06	Feedback	6	-0.80
0	3	107	2024-04-11	Feedback	None	-0.24
0	4	13	2025-01-05	Feedback	8	-0.36
0	5	289	2024-09-17	Inquiry	1	-0.12

```
In [ ]: # Save local copy of extracted Support DataFrame
support_df.to_pickle('support.pkl')
```

```
In [459... # Retrieve local copy of Support DataFrame
support_df = pd.read_pickle('support.pkl')
```

Transformation

Transform Flat File

```
In [217... # DATA CLEANING
```

```
In [487... # Data Cleaning - Online DataFrame
# Examine data using info(), nunique(), and pd.unique. The dataset has 100,000 records.
online_df.info()
online_df.nunique()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100000 entries, 0 to 99999
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   customer_id            100000 non-null  int64
1   purchase_date          100000 non-null  object
2   product_id             100000 non-null  int64
3   quantity               100000 non-null  int64
4   price                  100000 non-null  float64
5   payment_method         100000 non-null  object
6   shipping_address       100000 non-null  object
7   website_source         100000 non-null  object
8   purchase_channel       100000 non-null  object
dtypes: float64(1), int64(3), object(5)
memory usage: 6.9+ MB
```

```
Out[487... customer_id      500
purchase_date    366
product_id       10
quantity         3
price            61831
payment_method    3
shipping_address  100
website_source    3
purchase_channel  1
dtype: int64
```

```
In [489... online_df.apply(pd.unique)
```

```
Out[489... customer_id      [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14...
purchase_date    [2024-03-05, 2024-03-14, 2024-04-03, 2024-04-0...
product_id       [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
quantity         [2, 1, 3]
price            [834.5, 915.03, 320.26, 356.52, 621.0, 163.69,...
payment_method    [PayPal, Credit Card, Apple Pay]
shipping_address  [Address 92, Address 28, Address 94, Address 2...
website_source    [Paid Ad, Organic, Referral]
purchase_channel  [online]
dtype: object
```

```
In [491... # Display missing values - there are none
online_df[online_df.isnull().any(axis=1)]
```

```
Out[491... customer_id  purchase_date  product_id  quantity  price  payment_method  shipping_address  website
```



```
In [225... # Drop or Fill missing values if any. Coded for possible future use.
#online_df = online_df.dropna()
#online_df = online_df.ffill()
```

```
In [493... # Remove duplicates if any
online_df = online_df.drop_duplicates(keep='first')
```

```
In [495... # Add purchase channel column
online_df['purchase_channel'] = 'online'
```

```
In [497... # Change date column name
online_df = online_df.rename(columns={'order_date': 'purchase_date'})
```

```
In [233... # Display results of cleaning using head(), info(), nunique(), and pd.unique
online_df.head()
```

```
Out[233... customer_id  purchase_date  product_id  quantity  price  payment_method  shipping_address  webs
```

	customer_id	purchase_date	product_id	quantity	price	payment_method	shipping_address	webs
0	1	2024-03-05	1	2	834.50	PayPal	Address 92	
1	1	2024-03-05	1	1	915.03	PayPal	Address 28	
2	1	2024-03-14	1	2	320.26	PayPal	Address 94	
3	1	2024-04-03	1	1	356.52	Credit Card	Address 2	
4	1	2024-04-04	1	1	621.00	PayPal	Address 99	



In [235...

```
online_df.info()
online_df.nunique()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100000 entries, 0 to 99999
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   customer_id           100000 non-null  int64  
1   purchase_date          100000 non-null  object  
2   product_id             100000 non-null  int64  
3   quantity               100000 non-null  int64  
4   price                  100000 non-null  float64 
5   payment_method         100000 non-null  object  
6   shipping_address       100000 non-null  object  
7   website_source         100000 non-null  object  
8   purchase_channel       100000 non-null  object  
dtypes: float64(1), int64(3), object(5)
memory usage: 6.9+ MB
```

Out[235...

```
customer_id           500
purchase_date         366
product_id            10
quantity              3
price                61831
payment_method         3
shipping_address      100
website_source         3
purchase_channel       1
dtype: int64
```

In [237...

```
online_df.apply(pd.unique)
```

Out[237...

```
customer_id           [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14...
purchase_date         [2024-03-05, 2024-03-14, 2024-04-03, 2024-04-0...
product_id            [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
quantity              [2, 1, 3]
price                [834.5, 915.03, 320.26, 356.52, 621.0, 163.69,...
payment_method         [PayPal, Credit Card, Apple Pay]
shipping_address       [Address 92, Address 28, Address 94, Address 2...
website_source         [Paid Ad, Organic, Referral]
purchase_channel       [online]
dtype: object
```

Transform SQLite Table

In [473...

```
# Data Cleaning - Instore DataFrame
# Examine data using info(), nunique(), and pd.unique. The dataset has 80,000 records.
instore_df.info()
instore_df.nunique()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 80000 entries, 0 to 79999
Data columns (total 7 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   customer_id           79833 non-null   float64
1   transaction_date       80000 non-null   object
2   product_id            80000 non-null   int64
3   quantity              80000 non-null   int64
4   price                 80000 non-null   int64
5   store_location        80000 non-null   object
6   payment_method        80000 non-null   object
dtypes: float64(1), int64(3), object(3)
memory usage: 4.3+ MB

```

```

Out[473...] customer_id           500
            transaction_date  366
            product_id       10
            quantity         5
            price           481
            store_location    20
            payment_method     3
            dtype: int64

```

```

In [241...] instore_df.apply(pd.unique)

```

```

Out[241...] customer_id           [17.0, 331.0, 286.0, 394.0, 237.0, 65.0, 170.0...
            transaction_date      [2024-08-06, 2025-02-15, 2024-10-29, 2024-05-1...
            product_id            [2, 5, 1, 7, 3, 6, 8, 10, 4, 9]
            quantity              [5, 2, 4, 3, 1]
            price                [63, 462, 304, 125, 58, 372, 413, 222, 365, 37...
            store_location        [Store 3, Store 14, Store 2, Store 20, Store 1...
            payment_method        [Debit Card, Credit Card, Cash]
            dtype: object

```

```

In [475...] ''' Display missing values
167 out of 80,000 customer records have null customer IDs = 0.21%. Any relationship of these r
records is unknown. Therefore it was decided to drop these records.'''
instore_df[instore_df.isnull().any(axis=1)]

```

Out[475...

	customer_id	transaction_date	product_id	quantity	price	store_location	payment_method	
	112	NaN	2024-12-06	10	5	235	Store 20	Cash
	903	NaN	2025-02-20	6	2	317	Store 14	Cash
	1062	NaN	2024-12-18	9	1	452	Store 2	Debit Card
	1079	NaN	2025-02-03	4	5	476	Store 12	Credit Card
	1427	NaN	2024-07-28	9	5	246	Store 16	Credit Card
	...	...	...	...	...	...	...	...
	75942	NaN	2024-12-23	10	5	134	Store 9	Credit Card
	77406	NaN	2024-09-27	5	2	129	Store 2	Cash
	78144	NaN	2025-01-08	2	1	272	Store 1	Credit Card
	79080	NaN	2024-05-01	7	3	106	Store 4	Credit Card
	79761	NaN	2024-05-03	2	2	119	Store 10	Debit Card

167 rows × 7 columns

In [477...

```
# Drop - drop rows with missing customer_id
instore_df = instore_df.dropna()
```

In [479...

```
# Convert Instore Customer ID from float to int
instore_df['customer_id'] = instore_df['customer_id'].astype(int)

# Convert Instore Price from int to float
instore_df['price'] = instore_df['price'].astype(float)
```

In [481...

```
# Remove duplicates if any
instore_df = instore_df.drop_duplicates(keep='first')
```

In [483...

```
# Add purchase channel column
instore_df['purchase_channel'] = 'instore'
```

In [485...

```
# Change date column names
instore_df = instore_df.rename(columns={'transaction_date': 'purchase_date'})
```

In [255...

```
# Data Cleaning - Instore DataFrame
# Display results of cleaning using head(), info(), nunique(), and pd.unique
instore_df.info()
instore_df.nunique()
```



```
<class 'pandas.core.frame.DataFrame'>
Index: 79833 entries, 0 to 79999
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   customer_id           79833 non-null  int32
1   purchase_date         79833 non-null  object
2   product_id            79833 non-null  int64
3   quantity              79833 non-null  int64
4   price                 79833 non-null  float64
5   store_location        79833 non-null  object
6   payment_method        79833 non-null  object
7   purchase_channel      79833 non-null  object
dtypes: float64(1), int32(1), int64(2), object(4)
memory usage: 5.2+ MB
```

```
Out[255...] customer_id           500
purchase_date         366
product_id            10
quantity              5
price                 481
store_location        20
payment_method         3
purchase_channel       1
dtype: int64
```

```
In [257...] instore_df.apply(pd.unique)
```

```
Out[257...] customer_id           [17, 331, 286, 394, 237, 65, 170, 181, 357, 29...
purchase_date         [2024-08-06, 2025-02-15, 2024-10-29, 2024-05-1...
product_id            [2, 5, 1, 7, 3, 6, 8, 10, 4, 9]
quantity              [5, 2, 4, 3, 1]
price                 [63.0, 462.0, 304.0, 125.0, 58.0, 372.0, 413.0...
store_location        [Store 3, Store 14, Store 2, Store 20, Store 1...
payment_method        [Debit Card, Credit Card, Cash]
purchase_channel      [instore]
dtype: object
```

Transform Web API File

```
In [535...] # Data Cleaning - Support DataFrame
# Examine data using info(), nunique(), and pd.unique. The dataset has 2000 records.
support_df.info()
support_df.nunique()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 2000 entries, 0 to 0
Data columns (total 6 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   id                    2000 non-null  int64
1   customer_id           2000 non-null  int64
2   interaction_date       2000 non-null  object
3   interaction_type       2000 non-null  object
4   product_id            1359 non-null  object
5   sentiment_score        2000 non-null  float64
dtypes: float64(1), int64(2), object(3)
memory usage: 109.4+ KB
```

```
Out[535... id                2000
customer_id         482
interaction_date     365
interaction_type      4
product_id           10
sentiment_score     201
dtype: int64
```

```
In [537... support_df.apply(pd.unique)
```

```
Out[537... id                [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14...
customer_id         [126, 346, 107, 13, 289, 83, 334, 225, 137, 15...
interaction_date     [2025-01-18, 2024-12-06, 2024-04-11, 2025-01-0...
interaction_type     [Feedback, Inquiry, Complaint, Return]
product_id           [3, 6, None, 8, 1, 10, 9, 4, 7, 2, 5]
sentiment_score     [0.87, -0.8, -0.24, -0.36, -0.12, 0.45, -0.72,...
dtype: object
```

```
In [539... ''' Display missing values. 641 out of 2000 rows have null Product IDs. These values cannot be
will be kept as they represent valid customer interactions.'''
support_df[support_df.isnull().any(axis=1)]
```

```
Out[539...      id  customer_id  interaction_date  interaction_type  product_id  sentiment_score
0      3          107      2024-04-11          Feedback      None          -0.24
0      8          225      2024-07-20          Inquiry      None          -0.94
0     10          158      2024-12-14          Inquiry      None          -0.90
0     12           65      2025-02-06          Complaint      None           0.57
0     14          258      2024-08-18          Return       None           0.89
...   ...          ...          ...          ...          ...          ...
0  1984          283      2024-09-09          Inquiry      None          -0.97
0  1986          157      2024-04-24          Inquiry      None          -0.37
0  1992          430      2024-09-02          Return       None           0.48
0  1993          492      2024-07-26          Complaint      None          -0.62
0  1996          494      2024-08-16          Complaint      None           0.66
```

641 rows × 6 columns

```
In [541... # Drop or Fill. Fill null product ID fields with 0. Then convert Product ID to Integer
support_df['product_id'] = support_df['product_id'].fillna(0)
# Convert Support Product ID from float to int
support_df['product_id'] = support_df['product_id'].astype(int)
```

```
C:\Users\cindo\AppData\Local\Temp\ipykernel_6008\1714563504.py:2: FutureWarning: Downcasting ob
ject dtype arrays on .fillna, .ffill, .bfill is deprecated and will change in a future version.
Call result.infer_objects(copy=False) instead. To opt-in to the future behavior, set `pd.set_op
tion('future.no_silent_downcasting', True)`
support_df['product_id'] = support_df['product_id'].fillna(0)
```

```
In [543... # Remove duplicates if any
support_df = support_df.drop_duplicates(keep='first')
```

```
In [545... # Add purchase channel column
support_df['purchase_channel'] = 'support'
```

```
In [547... # Drop redundant ID column which is not in the other 2 data sources.
support_df = support_df.drop(columns=['id'])
```

```
In [549... # Data Cleaning - Support DataFrame
# Display results of cleaning using info(), nunique(), and pd.unique
support_df.info()
support_df.nunique()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 2000 entries, 0 to 0
Data columns (total 6 columns):
#   Column                Non-Null Count  Dtype
---  -
0   customer_id           2000 non-null   int64
1   interaction_date       2000 non-null   object
2   interaction_type       2000 non-null   object
3   product_id            2000 non-null   int32
4   sentiment_score       2000 non-null   float64
5   purchase_channel      2000 non-null   object
dtypes: float64(1), int32(1), int64(1), object(3)
memory usage: 101.6+ KB
```

```
Out[549... customer_id           482
interaction_date       365
interaction_type        4
product_id             11
sentiment_score       201
purchase_channel       1
dtype: int64
```

```
In [551... support_df.apply(pd.unique)
```

```
Out[551... customer_id           [126, 346, 107, 13, 289, 83, 334, 225, 137, 15...
interaction_date       [2025-01-18, 2024-12-06, 2024-04-11, 2025-01-0...
interaction_type       [Feedback, Inquiry, Complaint, Return]
product_id            [3, 6, 0, 8, 1, 10, 9, 4, 7, 2, 5]
sentiment_score       [0.87, -0.8, -0.24, -0.36, -0.12, 0.45, -0.72,...
purchase_channel      [support]
dtype: object
```

Join the 3 Datasets

```
In [277... # DATA INTEGRATION
```

```
In [553... ''' Join the 3 DataFrames by Customer ID and Product ID. This is effectively a concatenation o
3 DataFrames have primarily unique columns.'''
purchases_df = pd.concat([online_df, instore_df, support_df])
```

```
In [555... # DATA ENRICHMENT
''' Calculation of Total Purchase Value as the product of Quantity and Price is an example of
stored in the data warehouse.'''
purchases_df['total_purchase_value'] = purchases_df['quantity'] * purchases_df['price']
```

```
In [557... # Fill null Purchase Quantity values with 0.
purchases_df['quantity'] = purchases_df['quantity'].fillna(0)
# Then convert Purchase Quantity from float to Integer
purchases_df['quantity'] = purchases_df['quantity'].astype(int)
```

```
In [559... '''Verify the results of the join operation using head(), info(), nunique(), and pd.unique. Th
181,833 rows.'''
purchases_df.head()
```

```
Out[559...      customer_id  purchase_date  product_id  quantity  price  payment_method  shipping_address  webs
0           1      2024-03-05           1         2   834.50           PayPal      Address 92
1           1      2024-03-05           1         1   915.03           PayPal      Address 28
2           1      2024-03-14           1         2   320.26           PayPal      Address 94
3           1      2024-04-03           1         1   356.52      Credit Card      Address 2
4           1      2024-04-04           1         1   621.00           PayPal      Address 99
```

```
In [561... purchases_df.info()
purchases_df.nunique()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 181833 entries, 0 to 0
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype
---  -
0   customer_id            181833 non-null  int64
1   purchase_date          179833 non-null  object
2   product_id             181833 non-null  int64
3   quantity               181833 non-null  int32
4   price                  179833 non-null  float64
5   payment_method         179833 non-null  object
6   shipping_address       100000 non-null  object
7   website_source         100000 non-null  object
8   purchase_channel       181833 non-null  object
9   store_location         79833 non-null   object
10  interaction_date        2000 non-null    object
11  interaction_type        2000 non-null    object
12  sentiment_score        2000 non-null    float64
13  total_purchase_value    179833 non-null  float64
dtypes: float64(3), int32(1), int64(2), object(8)
memory usage: 20.1+ MB
```

```
Out[561... customer_id            500
purchase_date          366
product_id             11
quantity               6
price                 62016
payment_method         5
shipping_address       100
website_source         3
purchase_channel       3
store_location         20
interaction_date       365
interaction_type        4
sentiment_score        201
total_purchase_value   77995
dtype: int64
```

```
In [563... purchases_df.apply(pd.unique)
```

```
Out[563...] customer_id      [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14...
purchase_date    [2024-03-05, 2024-03-14, 2024-04-03, 2024-04-0...
product_id       [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 0]
quantity         [2, 1, 3, 5, 4, 0]
price            [834.5, 915.03, 320.26, 356.52, 621.0, 163.69,...
payment_method   [PayPal, Credit Card, Apple Pay, Debit Card, C...
shipping_address [Address 92, Address 28, Address 94, Address 2...
website_source   [Paid Ad, Organic, Referral, nan]
purchase_channel [online, instore, support]
store_location   [nan, Store 3, Store 14, Store 2, Store 20, St...
interaction_date  [nan, 2025-01-18, 2024-12-06, 2024-04-11, 2025...
interaction_type  [nan, Feedback, Inquiry, Complaint, Return]
sentiment_score  [nan, 0.87, -0.8, -0.24, -0.36, -0.12, 0.45, -...
total_purchase_value [1669.0, 915.03, 640.52, 356.52, 621.0, 327.38...
dtype: object
```

```
In [565...] # Save Local copy of Purchases DataFrame
purchases_df.to_pickle('purchases.pkl')
```

```
In [567...] # Retrieve Local copy of Support DataFrame
purchases_df = pd.read_pickle('purchases.pkl')
```

Aggregate the Joined Detail Data into a Summary DataFrame

```
In [569...] ''' GROUP BY customer ID and product ID. Calculate aggregate statistics (quantity and total pu
Note that other aggregations can be envisioned. This is just one example.'''
grouped_purchases_df = purchases_df.groupby(['customer_id', 'product_id'], as_index=False)[['q
'total_purchase_value']].sum()
```

```
In [571...] # Drop groups where product_id = null or 0 as these rows will not be useful in this particular
grouped_purchases_df = grouped_purchases_df[grouped_purchases_df.product_id > 0]
```

```
In [573...] '''Calculate and store Product Average Price = Total Purchase Value / Total Quantity for each
a potentially useful derived attribute.'''
grouped_purchases_df['product_average_price'] = \
    round(grouped_purchases_df['total_purchase_value'] / grouped_purchases_df
```

```
In [575...] # Convert Grouped Purchase Quantity from float to int
grouped_purchases_df['quantity'] = grouped_purchases_df['quantity'].astype(int)
```

```
In [577...] ''' Display results of aggregation using head(), info(), nunique(), and pd.unique. There are 5
purchased all 10 products in this test case, so the aggregated DataFrame has 500 x 10 = 5000 r
grouped_purchases_df.head()
```

```
Out[577...]
```

	customer_id	product_id	quantity	total_purchase_value	product_average_price
1	1	1	101	37554.86	371.83
2	1	2	99	36602.13	369.72
3	1	3	97	40301.93	415.48
4	1	4	96	43270.47	450.73
5	1	5	82	29975.06	365.55

```
In [579...] grouped_purchases_df.info()
grouped_purchases_df.nunique()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 5000 entries, 1 to 5353
Data columns (total 5 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   customer_id            5000 non-null   int64
1   product_id             5000 non-null   int64
2   quantity               5000 non-null   int32
3   total_purchase_value    5000 non-null   float64
4   product_average_price   5000 non-null   float64
dtypes: float64(2), int32(1), int64(2)
memory usage: 214.8 KB
```

```
Out[579...] customer_id            500
           product_id            10
           quantity            106
           total_purchase_value  4995
           product_average_price 4295
           dtype: int64
```

```
In [581...] grouped_purchases_df.apply(pd.unique)
```

```
Out[581...] customer_id            [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14...
           product_id            [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
           quantity            [101, 99, 97, 96, 82, 90, 69, 65, 54, 120, 78,...
           total_purchase_value [37554.86, 36602.13, 40301.93, 43270.47, 29975...
           product_average_price [371.83, 369.72, 415.48, 450.73, 365.55, 360.3...
           dtype: object
```

```
In [583...] # Save Local copy of Grouped Purchases DataFrame
           grouped_purchases_df.to_pickle('grouped_purchases.pkl')
```

```
In [585...] # Retrieve Local copy of Grouped Purchases DataFrame
           grouped_df = pd.read_pickle('grouped_purchases.pkl')
```

Loading

Create MySQL Tables

```
In [434...] ''' CREATE DATABASE / SCHEMA - done in MySQL directly as follows:
           CREATE SCHEMA IF NOT EXISTS 'etl';
           USE 'etl'; '''

           # Create Sqlalchemy engine
           connection_string = "mysql+mysqlconnector://root:chris@localhost:3306/etl"
           engine = create_engine(connection_string)
```

```
In [147...] # CREATE TABLES
           # Create Grouped Purchases table. If table already exists, first DROP table in MySQL.
           with engine.connect() as conn:
               conn.execute(text('CREATE TABLE IF NOT EXISTS grouped_purchases ( \
               customer_id            INT NOT NULL,\
               product_id             INT NOT NULL,\
               quantity               INT,\
               price                   NUMERIC(8,2),\
               total_purchase_value    NUMERIC(10,2),\
               product_average_price  NUMERIC(10,2),\
               PRIMARY KEY (customer_id, product_id)\
               )'))
```

In [201... *# Create Purchases table. If table already exists, first DROP table in MySQL.*

```
with engine.connect() as conn:
    conn.execute(text('CREATE TABLE IF NOT EXISTS purchases ( \
        customer_id          INT,\
        product_id           INT,\
        purchase_date         DATE,\
        purchase_channel      CHAR(8),\
        quantity              INT,\
        price                  NUMERIC(8,2),\
        total_purchase_value  NUMERIC(10,2),\
        payment_method        VARCHAR(12),\
        store_location        VARCHAR(10),\
        website_source        VARCHAR(10),\
        interaction_date       DATE,\
        interaction_type       VARCHAR(10),\
        sentiment_score       NUMERIC(6,2),\
        shipping_address       VARCHAR(12),\
        product_category      VARCHAR(10),\
        customer_segment      VARCHAR(10)\
    )'))
```

Insert Data into MySQL Tables

In [189... *'''Insert Grouped Purchases DataFrame into MySQL.*
To rerun, select appropriate value of if_exists, 'fail', 'replace', or 'append'.'''
grouped_purchases_df.to_sql(name='grouped_purchases', con=engine, schema='etl', if_exists='app
method='multi')

Out[189... 5000

In [295... *'''Insert Purchases DataFrame into MySQL*
To rerun, select appropriate value of if_exists, 'fail', 'replace', or 'append'.'''
purchases_df.to_sql(name='purchases', con=engine, schema='etl', if_exists='append', index=False

Out[295... 181833

Verify the Loading Process

In [359... with engine.connect() as conn:
 print('Rows in Grouped Purchases Table:\n',pd.read_sql_query('SELECT COUNT(*) FROM grouped
 print('Rows in Purchases Table:\n', pd.read_sql_query('SELECT COUNT(*) FROM purchases', co
 print('First 5 rows of Grouped Purchases Table:\n', pd.read_sql_query('SELECT * FROM group
 print('First 5 rows of Purchases Table:\n', pd.read_sql_query('SELECT * FROM purchases LIM

Rows in Grouped Purchases Table:

```
COUNT(*)
0      5000
```

Rows in Purchases Table:

```
COUNT(*)
0     181833
```

First 5 rows of Grouped Purchases Table:

	customer_id	product_id	quantity	price	total_purchase_value \
0	1	1	101	None	37554.86
1	1	2	99	None	36602.13
2	1	3	97	None	40301.93
3	1	4	96	None	43270.47
4	1	5	82	None	29975.06

	product_average_price
0	371.83
1	369.72
2	415.48
3	450.73
4	365.55

First 5 rows of Purchases Table:

	customer_id	product_id	purchase_date	purchase_channel	quantity	price \
0	1	1	2024-03-05	online	2	834.50
1	1	1	2024-03-05	online	1	915.03
2	1	1	2024-03-14	online	2	320.26
3	1	1	2024-04-03	online	1	356.52
4	1	1	2024-04-04	online	1	621.00

	total_purchase_value	payment_method	store_location	website_source \
0	1669.00	PayPal	None	Paid Ad
1	915.03	PayPal	None	Paid Ad
2	640.52	PayPal	None	Paid Ad
3	356.52	Credit Card	None	Paid Ad
4	621.00	PayPal	None	Organic

	interaction_date	interaction_type	sentiment_score	shipping_address \
0	None	None	None	Address 92
1	None	None	None	Address 28
2	None	None	None	Address 94
3	None	None	None	Address 2
4	None	None	None	Address 99

	product_category	customer_segment
0	None	None
1	None	None
2	None	None
3	None	None
4	None	None

Analytical Query Examples

```
In [426... # Total quantity and value of all online and instore purchases. Combines data from both online
with engine.connect() as conn:
    total_quantity = pd.read_sql_query('SELECT SUM(quantity) FROM grouped_purchases', conn)
    total_value = pd.read_sql_query('SELECT SUM(total_purchase_value) FROM grouped_purchases',
    print(f'Total items purchased: {total_quantity.iloc[0,0]:,.0f}')
    print(f'Total value of items purchased: ${total_value.iloc[0,0]:,.2f}')
```

Total items purchased: 440,061

Total value of items purchased: \$167,652,330.12


```
In [440]: '''This query combines data from all 3 original sources. It displays the total purchase value
score for each product.'''
with engine.connect() as conn:
    purchase_query_df = pd.read_sql_query('SELECT * FROM purchases', conn)
    grouped_product_df = purchase_query_df.groupby('product_id', as_index=False)
    average_sentiment_df = grouped_product_df.agg({'total_purchase_value': 'sum', 'sentiment_score'
average_sentiment_df.style.format(precision=2, thousands=',')
```

```
Out[440]:
```

	product_id	total_purchase_value	sentiment_score
0	0	0.00	-0.01
1	1	16,841,077.20	0.03
2	2	16,797,461.43	0.06
3	3	16,998,859.11	-0.03
4	4	16,867,532.10	0.01
5	5	16,871,236.09	-0.01
6	6	16,700,842.84	-0.10
7	7	16,645,437.04	0.00
8	8	16,864,507.79	0.00
9	9	16,524,472.87	0.06
10	10	16,540,903.65	-0.05

Summary

The purpose and goals of the project have been met within a reasonable time frame. Data can be extracted from diverse sources, transformed, combined into denormalized tables, and stored in a data warehouse that can create new value for a business or organization.

Appendix - File and Table Schemas

```
In [ ]: # Close database connection
with engine.connect() as conn:
    conn.close()
```

1. Input - Online Sales Data – 100,000 records

customer_id order_date product_id quantity price payment_method shipping_address website_source

2. Input - In-Store Transaction Data – 80,000 records

customer_id transaction_date product_id quantity price store_location payment_method

3. Input - Customer Support Interaction Data – 2000 records

customer_id interaction_date interaction_type product_id sentiment_score

4. Output – Denormalized Purchases Table - 181,833 rows

Sources

customer_id INTEGER Online, Instore, Support product_id INTEGER Online, Instore, Support
purchase_date DATE Online, Instore purchase_channel CHAR(8) Derived quantity INTEGER Online, Instore
price NUMERIC(8,2) Online, Instore total_purchase_value NUMERIC(10,2) Derived payment_method
VARCHAR(10) Online, Instore store_location VARCHAR(10) Instore website_source VARCHAR(10) Online
interaction_date DATE Support interaction_type VARCHAR(10) Support sentiment_score NUMERIC(6,2)
Support shipping_address VARCHAR(12) Online

5. Output – Grouped Purchases Table - 5000 rows

Sources

customer_id INTEGER Online, Instore product_id INTEGER Online, Instore total_quantity INTEGER Derived
total_purchase_value NUMERIC(10,2) Derived product_average_price NUMERIC(10,2) Derived

In []: